

Mediverse VR

The new JSON input(DefaultInteractionLog.json) format reduces symptoms for points 2 and 3 because it matches the formatter's expectations, but will reappear the moment we use a different log schema/vocabulary CombinedToolLog.json

1. Is each log entry a discrete surgical action? (Final_Biomistral.ipynb)

What the code infers

- count = 5
- grouped = True

What the generated sentence becomes

"Multiple incisions made on right foot using scalpel (5 cuts)"

Fix

Option 1. duration = last_timestamp - first_timestamp

Option 2(best) If the **Unity** logger can be updated to emit:

- cutting_end when the scalpel stops contacting / trigger released
- or "tool_active: true/false" events
- Then we can differentiate between **cutting_start** and **cutting_end** and only count **completed incision cycles**, not log frequency.

2. Formatted medical steps have words like "Scalpel_incise" and default fallback for target: "anatomical structure"

Why Scalpel_incise and anatomical structure slip through

- "Scalpel_incise".lower() → "scalpel_incise"
- It **does not contain**:
 - "cutting"
 - "drilling"
 - "drawing"

So the formatter:

- does **not understand** this action
- treats it as unknown
- and either:
 - prints it raw in grouped form, or
 - falls back to "Surgical procedure on ..."

JSON entries often:

- don't have target

- or use values **not present** in anatomy_map

So encode_anatomy() falls back to "anatomical structure"

Fix

1. Add an **action normalization layer**, just like tool_map.

Example:

```
self.action_map = {
    "scalpel_incise": "cutting",
    "drill_insert": "drilling",
    "oscillatingsaw_cut": "cutting"
}
```

Then normalize before interpretation:

```
raw_action => entry.get("action", "").lower()
```

```
action = self.action_map.get(raw_action, raw_action)
```

2. For anatomical structure: add a target normalization layer

Before encode_anatomy(), normalize target by removing Unity noise:

- drop anything after (
- drop "Instance", "Clone", "Fixed", etc.
- trim whitespace, lowercase

This would turn:

- "footr (Fixed) Instance(Clone)" → "footr" ScalpelInteractionLog
so encode_anatomy() can map it to "right foot".

3. (Subsequent cell tries to detect these potential hallucination) prompt_out has multiple "surgical procedure was done" (fails to detect action or tool)

Where those generic lines come from

They're produced by the formatter's **fallback** path in convert_to_medical_step() when it can't interpret the action field (like Scalpel_incise, Drill_insert, oscillatingsaw...) as "cutting/drilling/drawing". In that case, it defaults to:

- "Surgical procedure on {anatomy} using {tool}"

Fix

Improve the **pre-LLM formatting**, not the post-processing:

- **Action normalization:** map Scalpel_incise → cutting, Drill_insert → drilling, etc.

Then your step text becomes "Incision made..." / "Drilling performed..." instead of "Surgical procedure..."

- **Target normalization:** strip Unity object-name noise so anatomy resolves to "right foot / metatarsal..." instead of "anatomical structure".
- **Better grouping/deduping:** collapse repeated unknown actions into one summarized step (or suppress them).

4. **create_simple_narration_json is less robust- if JSON uses Items or is already a list, the function will fail**

But create_simple_narration_json is **not the "real pipeline"** — it's a simple, deterministic fallback that doesn't use the LLM at all

Assumes JSON is shaped like {"data": [{ "timestamp": ... }, ...]} (Cell 1 handled formatting better)

5. **Potential issue in map_sentences_sequential(): no. of narration sentences might not be equal to number of timestamps(log entries)**

[WARNING] 30 sentences vs 31 timestamps

- timestamps come from enhanced_CombinedToolLog.json (**one per log entry**)
- sentences come from prompt_out (**a paragraph summary**)

Timestamps-Narration count from enhanced_CombinedToolLog.json **!=** BioMistral narration count

Issues with current model (BioMistral):

- Model doesn't work well for large JSON inputs
- memory issues

Fix #1: Chunked Generation (Most Important)

Core idea

Instead of: 100 steps → 1 prompt

Do: 100 steps → 10 chunks of 10 steps → 10 narrations

Then concatenate.

Fix #2: Streaming / incremental processing

Instead of holding everything in memory:

- process chunk
- write partial enhanced JSON to disk
- free memory
- move on

This turns:

- memory explosion → constant memory footprint

Other model suggestions:

1. Qwen2.5 Instruct (7B / 14B)

Pros

- Very capable on structured, long inputs; the HF model card notes config supports 32,768 tokens and discusses long-text handling. [Hugging Face](#)
- License on HF is Apache-2.0. [Hugging Face](#)

Cons

- Not medically specialized, so clinical phrasing may be slightly less “native” than BioMistral — but our formatter already injects medical terms, which reduces that disadvantage.

Best when:

- #1 issue is large inputs and stability.

2. Mistral 7B Instruct v0.3

Pros

- Excellent instruction-following, compact and fast.
- HF lists it as Apache-2.0. [Hugging Face](#)

Con

- Not biomedical-specialized like BioMistral.

Best when:

- We want a simple, reliable baseline with permissive licensing.

3. BioMistral 7B

Pros

- Biomedical specialization, competitive medical QA performance per its model card. [Hugging Face](#)
- Config shows max_position_embeddings: 32768 (so it *can* be set up for long context) [Hugging Face](#)

Con

- large-input issues(that we have right now) is likely due to prompt length + generation strategy + memory rather than BioMistral being “wrong.”

Best when:

- We want the best clinical tone.

Overall best: BioMistral